DUNGEONS – Predicting RPG Dungeon Difficulty

Machine Learning I & II – Master in Data Science, University of Seville

Miguel García Moreno

2026-11-02

Table of contents

Introduction and AI-use declaration **3**

1 Data preparation **4**
1.1 Loading and initial exploration . . . 4
1.2 Variable types and target encoding . . . 6
1.3 Predictor selection . . . 6
1.4 Train / test split (75 % / 25 %) . . . 6
1.5 Scaling (for distance/gradient-based models) . . . 7

2 CART, Random Forest, and Neural Network **8**
2.1 CART . . . 8
2.2 Random Forest . . . 11
2.3 Neural network (one hidden layer, `nnet`) . . . 14
2.4 Model comparison . . . 15

3 Principal Component Analysis (PCA) **17**
3.1 Explained variance . . . 17
3.2 Variable loadings . . . 18
3.3 Biplot . . . 19

4 Genetic Algorithm for Feature Selection **21**

5 Dichotomous variable `DIFICIL` **23**
5.1 Class distribution . . . 23

6 Radial SVM for `DIFICIL` **25**

7 The Added Problem **27**
7.1 Diagnosis 1 – Zero / near-zero variance . . . 27
7.2 Diagnosis 2 – Multicollinearity . . . 28
7.3 Diagnosis 3 – Class imbalance . . . 29
7.4 Conclusion . . . 30

8 Balanced Classification **31**
8.1 Balanced SVM 31
8.2 Simple Perceptron 32
8.3 Comparison 33
8.4 Prediction for `ID = 40` 33

Reproducibility notes **35**

9 Transparency and responsible declaration **36**

Introduction and AI-use declaration

This report implements the full **DUNGEONS** assignment: predicting the ordinal difficulty `RANGO` of a fantasy dungeon-crawler level from the 3D positions and combat power of its five enemies, and later a binary "hard vs. not hard" (`DIFICIL`) classification.

Reproducibility. The random seed `1234` is fixed globally and re-set before every random operation. Helper functions live in `R/helpers.R` to keep the report focused on methodology and interpretation.

How to render. This is a Quarto document targeting PDF. It requires a working R installation, the packages listed in the setup chunk (installed automatically if missing) and a LaTeX engine (`tinytex::install_tinytex()` if you do not already have one). The data file `dungeons.csv` must be placed in the same folder as this `.qmd`.

> **AI-usage declaration (required by the assignment protocol).** A Large Language Model (Claude) was used as a support tool to help structure the code and draft explanations. Representative prompts included: *"Implement a modular R/Quarto ML report for the DUNGEONS dataset: CART, Random Forest and nnet for the multiclass RANGO target; PCA; GA feature selection; radial SVM for a DIFICIL binary target; detect the added data problem; and a balanced re-classification with SVM and a simple perceptron."* All AI-suggested code was reviewed, corrected and validated by the author, who is solely responsible for the final results. Any statement that depends on executing the code against the real `dungeons.csv` is left as an interpretation comment in the code and must be confirmed against the rendered output (these are marked `# FLAG:` where a human check is needed).

1 Data preparation

1.1 Loading and initial exploration

```
#> [1] 350  23
```

```
#> spc_tbl_ [350 x 23] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
#>  $ ID       : num [1:350] 1 2 3 4 5 6 7 8 9 10 ...
#>  $ N_ENEMIES: num [1:350] 5 5 5 5 5 5 5 5 5 5 ...
#>  $ RANGO    : chr [1:350] "Plata" "Bronce" "Bronce" "Bronce" ...
#>  $ 1_X      : num [1:350] 12.6 34.7 15.9 11 24 ...
#>  $ 1_Y      : num [1:350] 9.52 21.64 16.17 18.09 14.33 ...
#>  $ 1_Z      : num [1:350] 8 6.2 4.09 6.41 4.22 4.87 1.37 2.94 3.38 2.54 ...
#>  $ 1_PODER  : num [1:350] 17.4 7.44 19.52 32.75 47.91 ...
#>  $ 2_X      : num [1:350] 36.9 8.19 29.66 27.18 32.01 ...
#>  $ 2_Y      : num [1:350] 21.37 7.82 20.17 5.63 6.19 ...
#>  $ 2_Z      : num [1:350] 0.5 4.85 0.22 6.4 4.08 5.08 1.87 5.36 4.88 5.02 ...
#>  $ 2_PODER  : num [1:350] 28.8 5 25.9 12 62.5 ...
#>  $ 3_X      : num [1:350] 35.34 8.09 23.13 13.39 33 ...
#>  $ 3_Y      : num [1:350] 8.52 16.96 17.27 20 7.94 ...
#>  $ 3_Z      : num [1:350] 0.19 3.73 2.17 4.02 0.73 1.68 3.87 3.77 2.74 1.69 ...
#>  $ 3_PODER  : num [1:350] 34.7 5 22 10.9 63.5 ...
#>  $ 4_X      : num [1:350] 14.75 25.06 25.04 8.63 25.45 ...
#>  $ 4_Y      : num [1:350] 12.02 5.88 12.39 18.57 7.49 ...
#>  $ 4_Z      : num [1:350] 5.47 2.75 0.63 6.09 1.1 4.55 6.96 4.96 6.08 1.59 ...
#>  $ 4_PODER  : num [1:350] 35.4 5 40.2 34 65 ...
#>  $ 5_X      : num [1:350] 8.9 16.8 32.4 20.2 35 ...
#>  $ 5_Y      : num [1:350] 18.72 3.85 5.93 22.31 11.31 ...
#>  $ 5_Z      : num [1:350] 5.64 6.5 0.9 0.24 4.75 5.9 2.52 4.79 7.13 4.66 ...
#>  $ 5_PODER  : num [1:350] 35.67 5.53 30.4 14.37 64.7 ...
#>  - attr(*, "spec")=
#>   .. cols(
#>   ..   ID = col_double(),
#>   ..   N_ENEMIES = col_double(),
#>   ..   RANGO = col_character(),
#>   ..   `1_X` = col_double(),
#>   ..   `1_Y` = col_double(),
#>   ..   `1_Z` = col_double(),
#>   ..   `1_PODER` = col_double(),
#>   ..   `2_X` = col_double(),
#>   ..   `2_Y` = col_double(),
#>   ..   `2_Z` = col_double(),
#>   ..   `2_PODER` = col_double(),
#>   ..   `3_X` = col_double(),
#>   ..   `3_Y` = col_double(),
#>   ..   `3_Z` = col_double(),
#>   ..   `3_PODER` = col_double(),
```

```
#>   ..   `4_X` = col_double(),
#>   ..   `4_Y` = col_double(),
#>   ..   `4_Z` = col_double(),
#>   ..   `4_PODER` = col_double(),
#>   ..   `5_X` = col_double(),
#>   ..   `5_Y` = col_double(),
#>   ..   `5_Z` = col_double(),
#>   ..   `5_PODER` = col_double()
#>   .. )
#>  - attr(*, "problems")=<externalptr>

#>        ID           N_ENEMIES    RANGO               1_X
#>  Min.   :  1.00   Min.   :5   Length:350         Min.   : 3.38
#>  1st Qu.: 88.25   1st Qu.:5   Class :character   1st Qu.:12.38
#>  Median :175.50   Median :5   Mode  :character   Median :20.01
#>  Mean   :175.50   Mean   :5                      Mean   :20.35
#>  3rd Qu.:262.75   3rd Qu.:5                      3rd Qu.:27.82
#>  Max.   :350.00   Max.   :5                      Max.   :38.79
#>       1_Y              1_Z           1_PODER          2_X
#>  Min.   : 0.450   Min.   :0.000   Min.   : 5.00   Min.   : 2.85
#>  1st Qu.: 8.295   1st Qu.:1.417   1st Qu.:19.51   1st Qu.:14.26
#>  Median :12.835   Median :3.355   Median :34.15   Median :21.46
#>  Mean   :12.984   Mean   :3.403   Mean   :35.88   Mean   :21.29
#>  3rd Qu.:17.782   3rd Qu.:5.197   3rd Qu.:50.36   3rd Qu.:28.24
#>  Max.   :24.320   Max.   :8.000   Max.   :89.77   Max.   :39.67
#>       2_Y              2_Z           2_PODER          3_X
#>  Min.   : 1.230   Min.   :0.000   Min.   : 5.00   Min.   : 4.34
#>  1st Qu.: 7.332   1st Qu.:1.728   1st Qu.:24.19   1st Qu.:13.66
#>  Median :13.045   Median :3.230   Median :36.77   Median :21.82
#>  Mean   :12.597   Mean   :3.341   Mean   :38.70   Mean   :21.53
#>  3rd Qu.:17.150   3rd Qu.:4.890   3rd Qu.:52.31   3rd Qu.:28.56
#>  Max.   :23.710   Max.   :7.970   Max.   :87.52   Max.   :38.59
#>       3_Y              3_Z           3_PODER          4_X
#>  Min.   : 1.210   Min.   :0.000   Min.   : 5.00   Min.   : 1.55
#>  1st Qu.: 8.053   1st Qu.:1.860   1st Qu.:25.05   1st Qu.:13.83
#>  Median :12.485   Median :3.680   Median :39.95   Median :20.95
#>  Mean   :12.354   Mean   :3.634   Mean   :40.92   Mean   :21.00
#>  3rd Qu.:17.017   3rd Qu.:5.287   3rd Qu.:56.23   3rd Qu.:28.38
#>  Max.   :25.000   Max.   :8.000   Max.   :92.78   Max.   :39.57
#>       4_Y              4_Z           4_PODER          5_X
#>  Min.   : 1.290   Min.   :0.000   Min.   :  5.00   Min.   : 2.12
#>  1st Qu.: 8.435   1st Qu.:1.843   1st Qu.: 29.63   1st Qu.:14.04
#>  Median :13.350   Median :3.775   Median : 43.10   Median :20.89
#>  Mean   :12.988   Mean   :3.608   Mean   : 44.98   Mean   :21.24
#>  3rd Qu.:17.527   3rd Qu.:5.295   3rd Qu.: 60.36   3rd Qu.:27.84
#>  Max.   :24.450   Max.   :8.000   Max.   :108.34   Max.   :39.94
#>       5_Y              5_Z           5_PODER
#>  Min.   : 0.720   Min.   :0.000   Min.   :  5.00
```

```
#>  1st Qu.: 7.253   1st Qu.:2.033   1st Qu.: 31.19
#>  Median :12.330   Median :3.895   Median : 44.48
#>  Mean   :12.379   Mean   :3.688   Mean   : 46.08
#>  3rd Qu.:17.380   3rd Qu.:5.287   3rd Qu.: 59.49
#>  Max.   :23.520   Max.   :7.870   Max.   :111.46
```

```
#>        ID N_ENEMIES     RANGO       1_X       1_Y       1_Z   1_PODER       2_X
#>         0         0         0         0         0         0         0         0
#>       2_Y       2_Z   2_PODER       3_X       3_Y       3_Z   3_PODER       4_X
#>         0         0         0         0         0         0         0         0
#>       4_Y       4_Z   4_PODER       5_X       5_Y       5_Z   5_PODER
#>         0         0         0         0         0         0         0
```

The initial exploratory analysis finds no missing values. The statistical summary also confirms that N_ENEMIES is a constant variable and that ID is a unique identifier.

1.2 Variable types and target encoding

Some transformations are needed for correct variable typing. The target RANGO is encoded as an ordinal variable, making the order among categories explicit.

```
#> RANGO values not matching the expected levels: 0
```

```
#>
#>   Bronce    Plata      Oro  Platino Diamante  Maestro
#>       76       72       90       53       35       24
```

The target distribution is right-skewed.

1.3 Predictor selection

```
#>  [1] "1_X"     "1_Y"     "1_Z"     "1_PODER" "2_X"     "2_Y"     "2_Z"
#>  [8] "2_PODER" "3_X"     "3_Y"     "3_Z"     "3_PODER" "4_X"     "4_Y"
#> [15] "4_Z"     "4_PODER" "5_X"     "5_Y"     "5_Z"     "5_PODER"
```

1.4 Train / test split (75 % / 25 %)

To obtain adequate measures of generalisation ability later, a stratified sample is used for the train and test sets.

```
#>       Bronce Plata   Oro Platino Diamante Maestro
#> train  0.216 0.205 0.258   0.152    0.102   0.068
#> test   0.221 0.209 0.256   0.151    0.093   0.070
```

1.5 Scaling (for distance/gradient-based models)

For algorithms sensitive to scale, transformed train and test sets are built via standardisation. The same mean and variance used on the training set are applied to the test set. As a note, if the model is later put into production, new data must also be standardised with the same parameters used to build the training and test sets.

```
#>       1_X 1_Y 1_Z
#> mean    0   0   0
#> sd      1   1   1
```

2 CART, Random Forest, and Neural Network

Three classical models are fitted; first, a classification tree.

```
#> RANGO ~ `1_X` + `1_Y` + `1_Z` + `1_PODER` + `2_X` + `2_Y` + `2_Z` +
#>     `2_PODER` + `3_X` + `3_Y` + `3_Z` + `3_PODER` + `4_X` + `4_Y` +
#>     `4_Z` + `4_PODER` + `5_X` + `5_Y` + `5_Z` + `5_PODER`
```

2.1 CART

To prune the tree later, the full tree is grown first, achieving absolute purity. This tree is clearly overfit to the training set and has poor generalisation ability.

```
#>
#> Classification tree:
#> rpart::rpart(formula = model_formula, data = train_data, method = "class",
#>     control = rpart::rpart.control(cp = 0, minsplit = 2, minbucket = 1,
#>         xval = 10))
#>
#> Variables actually used in tree construction:
#>  [1] 1_PODER 1_X     1_Y     1_Z     2_PODER 2_X     2_Z     3_PODER 3_X
#> [10] 3_Y     4_PODER 4_X     4_Y     4_Z     5_PODER 5_X     5_Z
#>
#> Root node error: 196/264 = 0.74242
#>
#> n= 264
#>
#>           CP nsplit rel error  xerror     xstd
#> 1  0.2448980      0  1.000000 1.00000 0.036251
#> 2  0.0867347      1  0.755102 0.78571 0.040869
#> 3  0.0714286      2  0.668367 0.76531 0.041062
#> 4  0.0459184      3  0.596939 0.71939 0.041353
#> 5  0.0357143      5  0.505102 0.70408 0.041406
#> 6  0.0306122      6  0.469388 0.63776 0.041391
#> 7  0.0255102      7  0.438776 0.62755 0.041353
#> 8  0.0204082     10  0.362245 0.59694 0.041181
#> 9  0.0153061     11  0.341837 0.58163 0.041062
#> 10 0.0127551     14  0.295918 0.59184 0.041144
#> 11 0.0102041     19  0.229592 0.59184 0.041144
#> 12 0.0076531     29  0.122449 0.63265 0.041373
#> 13 0.0051020     33  0.091837 0.62755 0.041353
#> 14 0.0025510     49  0.010204 0.68878 0.041438
#> 15 0.0000000     53  0.000000 0.69388 0.041430
```

The fitted tree is shown below.

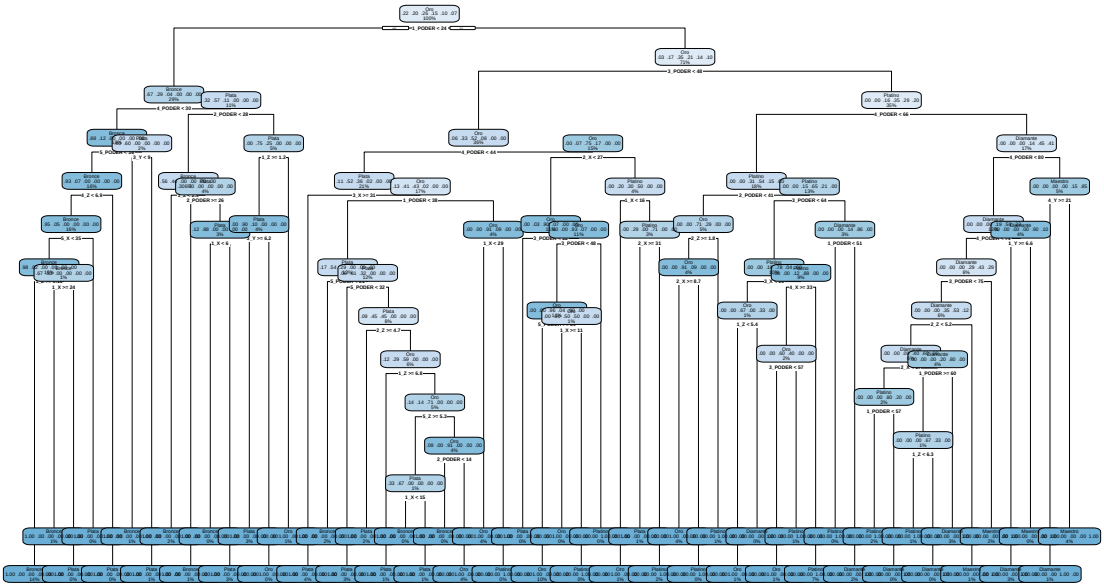


Figure 1: Fully grown CART tree (cp = 0). Expected to be large and overfit.

The tree is then pruned with the criterion set in the assignment.

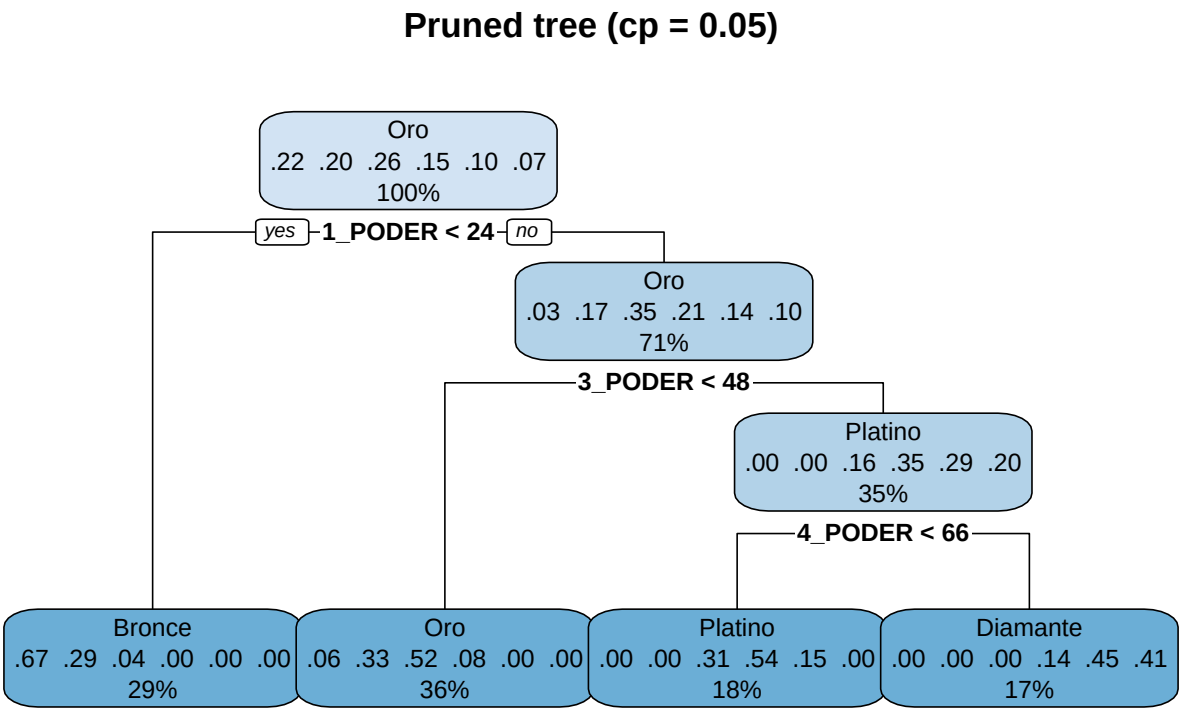


Figure 2: Pruned CART tree (cp = 0.05): smaller, more interpretable, less overfit.

This tree is the best possible that still meets the cp=0.05 criterion, because it comes from pruning the full tree (impurity = 0). Being very small, no further criteria are needed, as they would shrink an already compact tree. It also offers greater interpretability by simplifying the dichotomous logic of the first tree. This is the tree chosen by the author.

Table 1: CART test-set metrics

Model	Accuracy	Balanced_Accuracy	Precision	Recall	F1
CART (pruned)	0.477	0.658	0.301	0.425	0.352

Because the set is unbalanced, balanced accuracy matters more than overall accuracy. Here it is 65%, so further improvement is needed. Precision and recall are both low: among observations assigned to a given class the mean error rate is 0.301, and only 0.42 of the test data are classified into their true category.

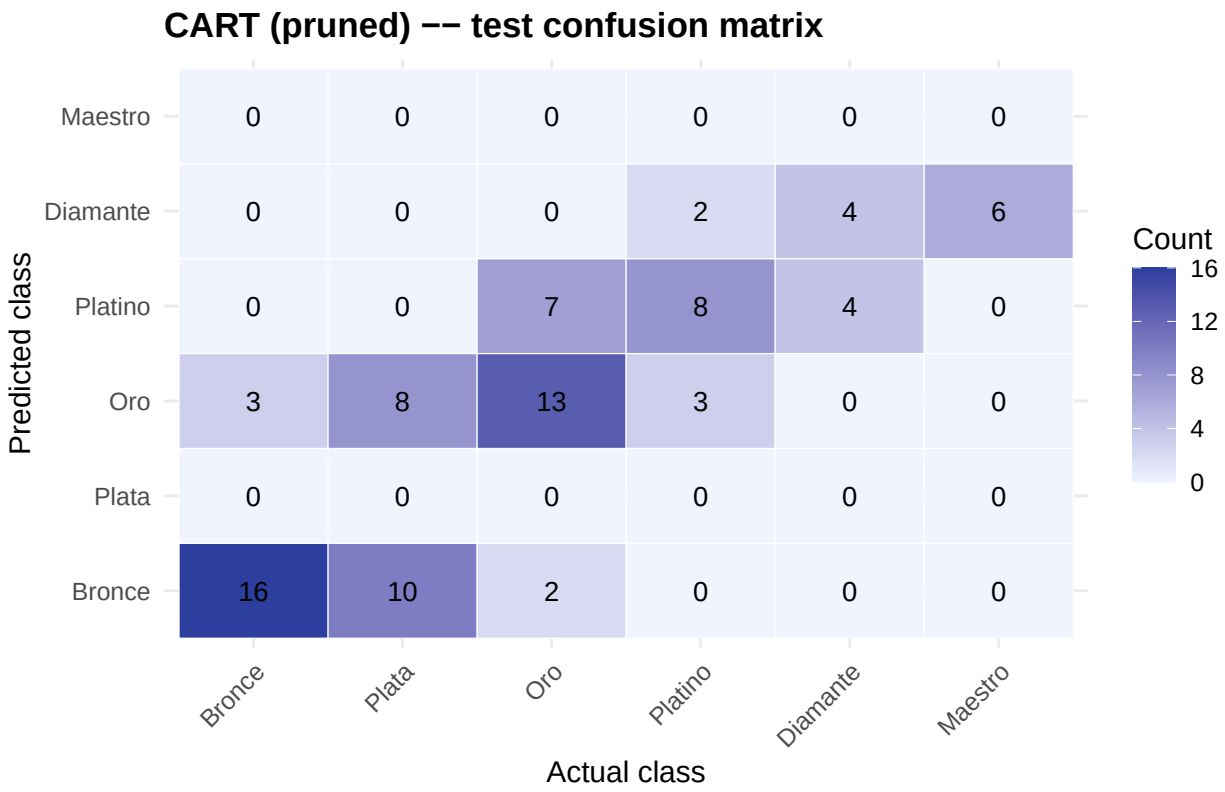


Figure 3: CART confusion matrix on the test set.

A dense diagonal indicates some learning. Notably, no Plata case is classified correctly—this is the tree's main problem and the primary cause of its low performance. Looking back at the "Pruned tree" figure explains this: no terminal node predicts Plata or Maestro.

2.2 Random Forest

The next model is Random Forest, fitted according to the assignment instructions. Tuning mtry specifically aims to improve predictive ability by forcing each tree to use at most mtry predictors.

```
#> Best Random Forest hyperparameters:

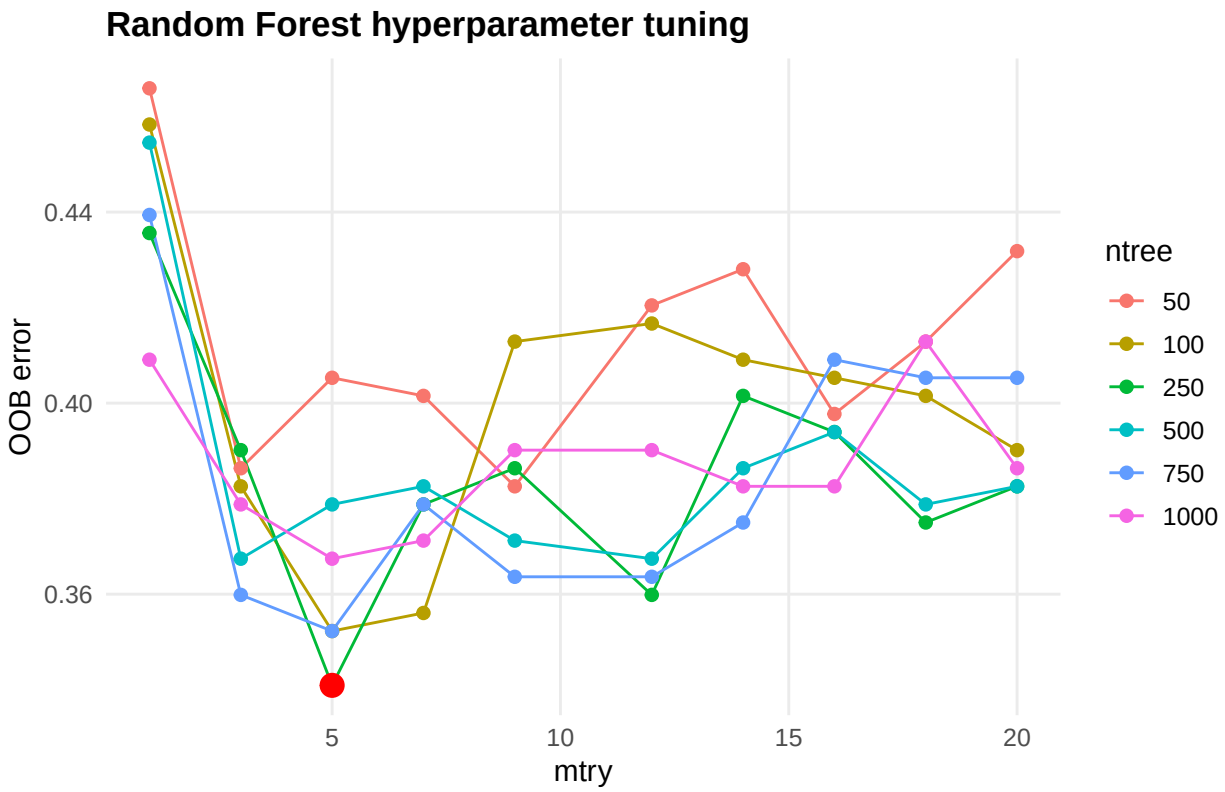
#>   mtry      : 5

#>   ntree     : 250

#>   OOB Error : 0.3409
```

After the grid search for the best hyperparameters, the model is fitted with them. The forest will use 5 variables in each tree. Because OOB data were used to select the best model, it still makes

sense to use the test set to evaluate generalisation ability, as those observations remain unseen. The grid search is visualised below.




```
#>
#> Call:
#>  randomForest(x = x_train, y = y_train, ntree = best_ntree, mtry = best_mtry,      importanc
#>                Type of random forest: classification
#>                      Number of trees: 250
#> No. of variables tried at each split: 5
#>
#>         OOB estimate of  error rate: 37.88%
#> Confusion matrix:
#>          Bronce Plata Oro Platino Diamante Maestro class.error
#> Bronce       44    13   0       0        0       0   0.2280702
#> Plata        10    29  15       0        0       0   0.4629630
#> Oro           0    13  48       7        0       0   0.2941176
#> Platino       0     0  13      23        4       0   0.4250000
#> Diamante      0     0   0      10       12       5   0.5555556
#> Maestro       0     0   0       0       10       8   0.5555556

#> Final OOB error estimate: 0.3788
```

The OOB error may look somewhat high, but true predictive ability will be checked later via the test-set confusion matrix.

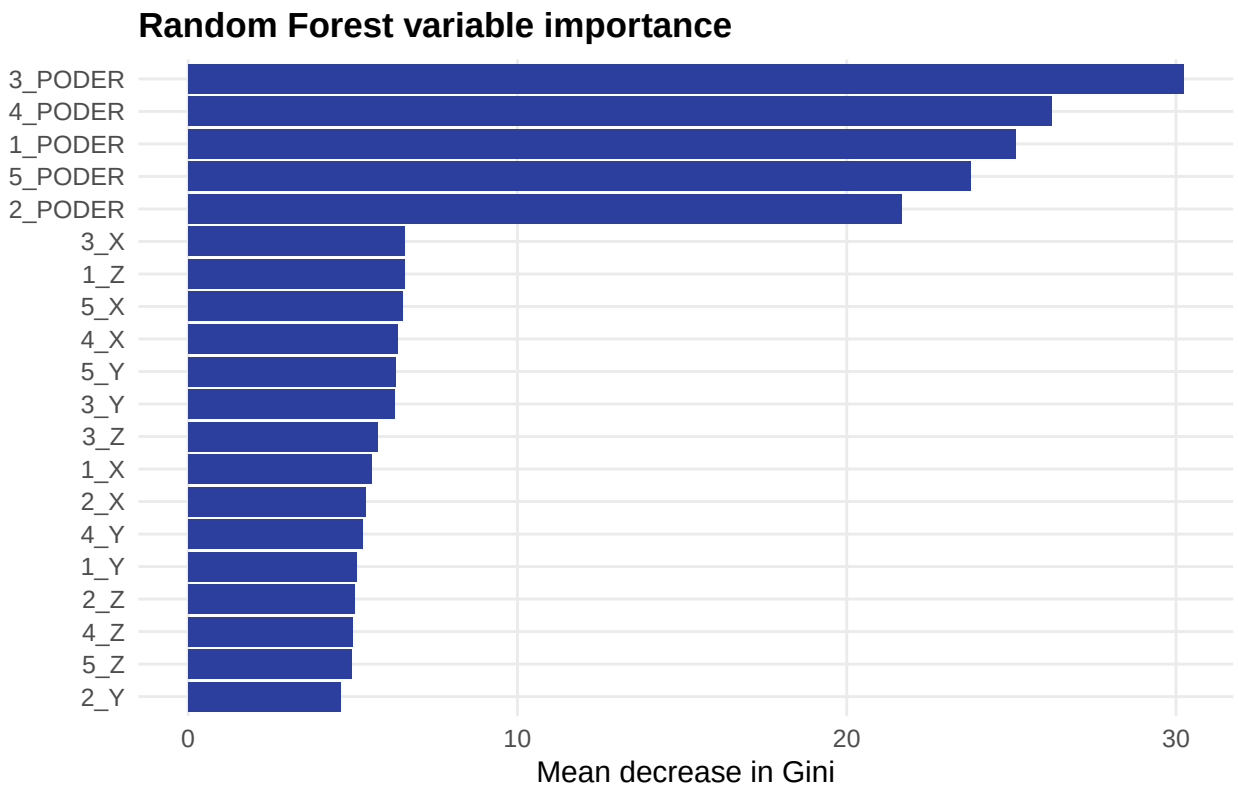


Figure 4: Variable importance (mean decrease in Gini impurity).

The variables that discriminate best in the random forest model can be seen above.

Table 2: Random Forest test-set metrics

Model	Accuracy	Balanced_Accuracy	Precision	Recall	F1
Random Forest	0.628	0.771	0.642	0.62	0.628

The metrics indicate a considerable improvement over CART.

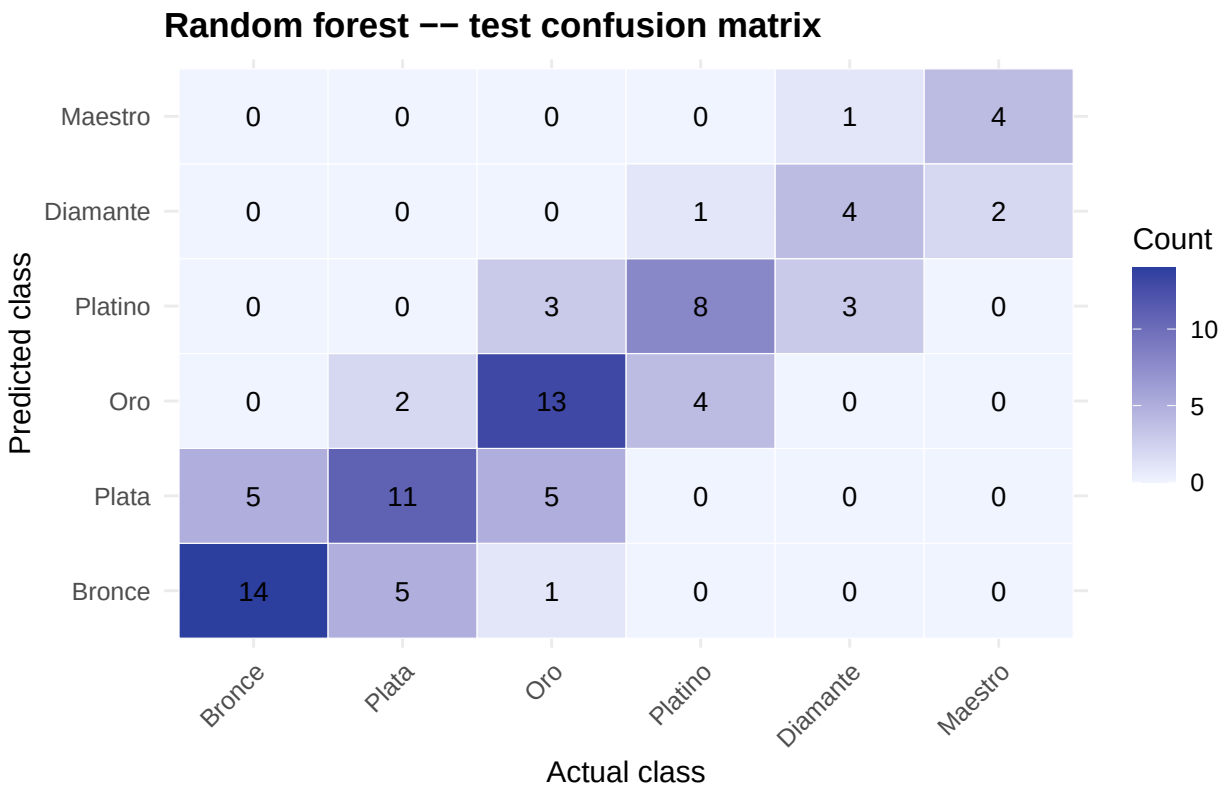


Figure 5: Random forest confusion matrix on the test set.

The Random Forest shows solid learning; notably, Plata and Maestro are now classified. Better results than CART were expected, since the forest builds many trees and votes on each prediction. It is therefore much more robust, at the cost of interpretability.

2.3 Neural network (one hidden layer, `nnet`)

Because neural networks are sensitive to scale, the scaled data set is used.

```
#> Selected hidden-layer size: 7 | decay: 0.01
```

Table 3: Neural-network CV accuracy by hidden size

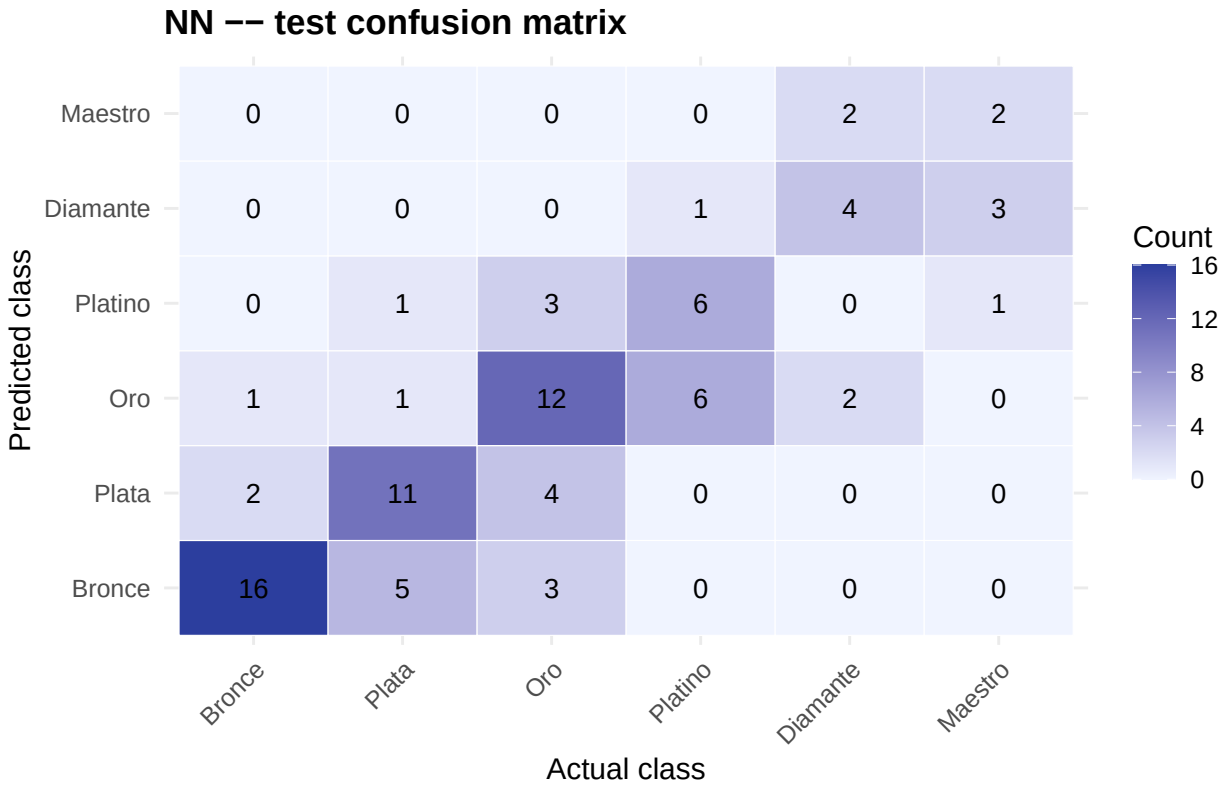
size	decay	Accuracy	Kappa
5	0.01	0.519	0.403
6	0.01	0.474	0.350
7	0.01	0.553	0.450
8	0.01	0.531	0.420

Based on the grid search, the best parameters are selected and the model is built with them.

Table 4: Neural-network test-set metrics

Model	Accuracy	Balanced_Accuracy	Precision	Recall	F1
Neural Network	0.593	0.732	0.567	0.549	0.553

Again the neural network outperforms CART, the first model considered.



Predictions appear in every category, and second-order mistakes (labels two ranks away) appear that were rare in the earlier models (CART has only one). The largest errors seem to occur at the highest levels—Maestro, Diamante and Platino. This small network has learned general patterns but still makes several mistakes; more neurons or a different decay might help.

2.4 Model comparison

Table 5: Multiclass model comparison on the test set

Model	Accuracy	Balanced_Accuracy	Precision	Recall	F1
CART (pruned)	0.477	0.658	0.301	0.425	0.352
Random Forest	0.628	0.771	0.642	0.620	0.628
Neural Network	0.593	0.732	0.567	0.549	0.553

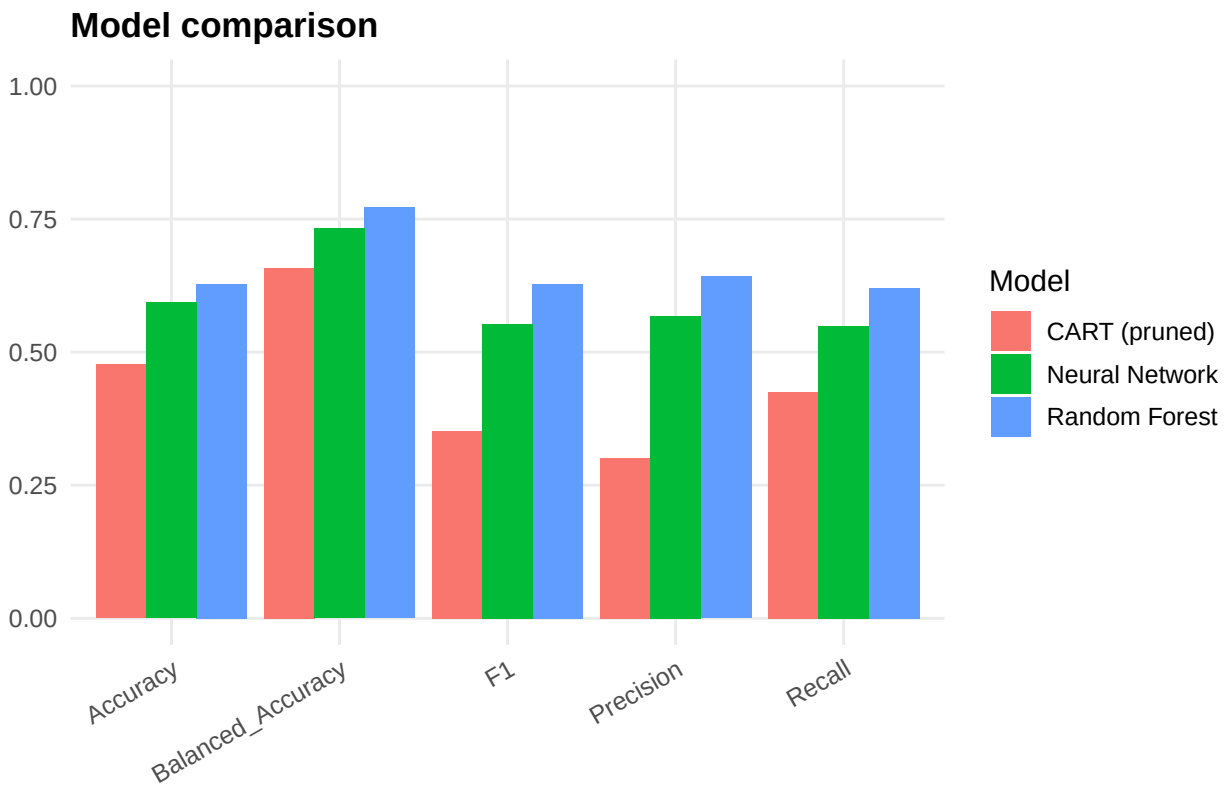


Figure 6: Test-set metric comparison across the three models.

Random Forest obtains the best results on every metric and would therefore be the chosen model, followed by the neural network and then CART, which is the most interpretable despite weaker results. Even so, choosing Random Forest is still somewhat premature: none of its metrics exceeds 0.8. Next steps would include a larger sample to give the forest more information, or more complex neural networks (which need more data).

3 Principal Component Analysis (PCA)

Scaled data are used for this method. PCA supports feature selection, because loadings on the principal components reveal which variables matter most. It is also a dimensionality-reduction method via orthogonal projections, used for example in LLM embeddings.

3.1 Explained variance

Table 6: PCA explained variance by component

PC	Eigenvalue	Prop_Variance	Cumulative
PC1	4.361	0.218	0.218
PC2	1.449	0.072	0.291
PC3	1.347	0.067	0.358
PC4	1.242	0.062	0.420
PC5	1.198	0.060	0.480
PC6	1.167	0.058	0.538
PC7	1.028	0.051	0.590
PC8	0.981	0.049	0.639
PC9	0.944	0.047	0.686
PC10	0.879	0.044	0.730
PC11	0.873	0.044	0.773
PC12	0.857	0.043	0.816
PC13	0.805	0.040	0.857
PC14	0.733	0.037	0.893
PC15	0.714	0.036	0.929
PC16	0.707	0.035	0.964
PC17	0.199	0.010	0.974
PC18	0.191	0.010	0.984
PC19	0.171	0.009	0.992
PC20	0.154	0.008	1.000

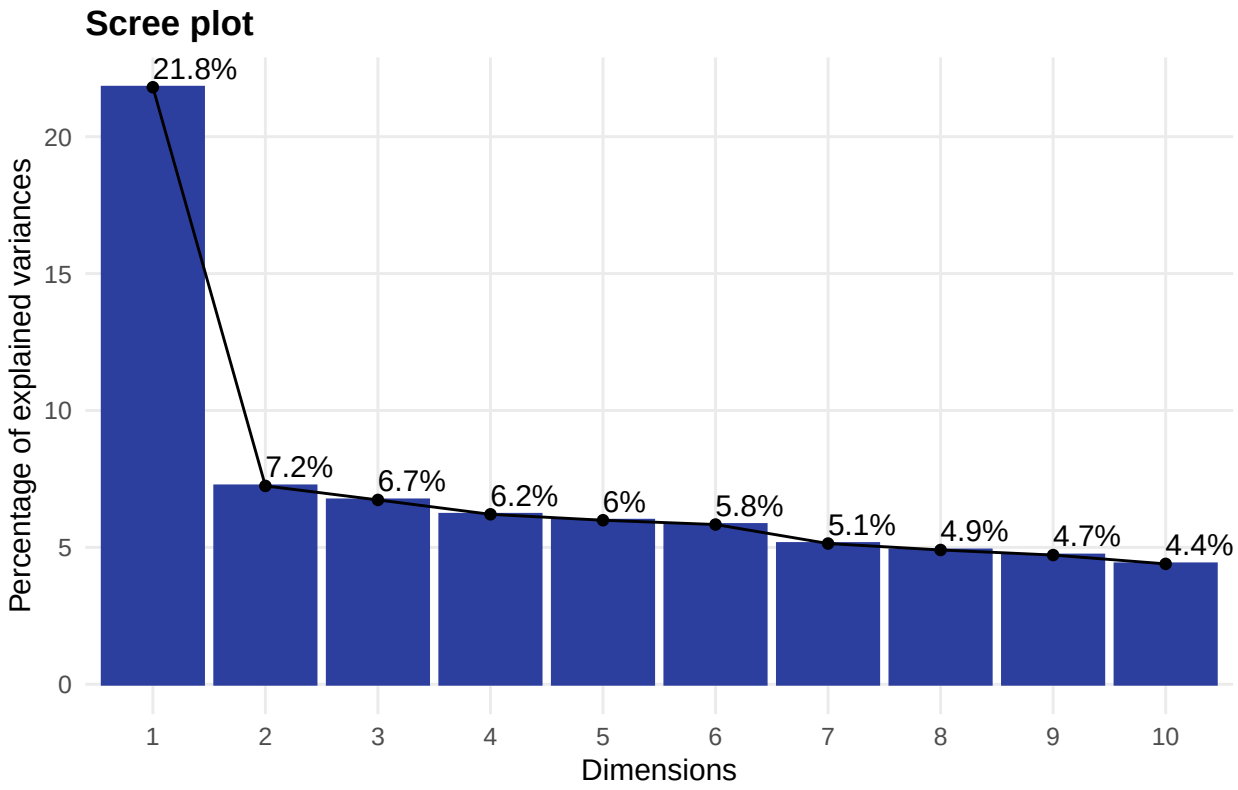


Figure 7: Scree plot: percentage of variance explained by each component.

For this sample, the variance explained by the principal components is modest. Ten components would be needed to explain 75% of the variance. The most influential variables on the first four components are examined next.

3.2 Variable loadings

Table 7: Variable loadings (first components)

	PC1	PC2	PC3	PC4
1_X	0.055	-0.202	0.042	0.129
1_Y	0.029	0.059	0.362	0.144
1_Z	-0.030	0.314	0.323	-0.111
1_PODER	0.434	0.056	-0.005	-0.028
2_X	-0.022	0.341	-0.006	0.538
2_Y	0.019	-0.269	0.301	0.411
2_Z	0.071	-0.223	0.193	0.246
2_PODER	0.438	0.074	-0.001	-0.026
3_X	0.127	-0.042	-0.104	0.279
3_Y	-0.014	-0.238	-0.463	0.092
3_Z	0.036	0.063	-0.035	-0.278
3_PODER	0.441	0.039	0.018	-0.039

	PC1	PC2	PC3	PC4
4_X	0.014	-0.110	0.274	-0.463
4_Y	-0.055	0.275	0.080	0.155
4_Z	-0.069	0.516	0.190	0.029
4_PODER	0.438	0.059	-0.049	-0.036
5_X	0.066	-0.352	0.215	0.060
5_Y	0.003	0.241	-0.368	-0.029
5_Z	0.011	-0.073	0.326	-0.135
5_PODER	0.439	0.049	-0.003	0.043

On PC1, (1_PODER, 2_PODER, …, 5_PODER) are highly influential, approaching 0.5; the rest are not noteworthy. On PC2, 5_X, 4_Z and 2_X matter more. A similar reading applies to the remaining components. Still, the loadings are not very strong: no variable even exceeds the 0.5 threshold.

3.3 Biplot

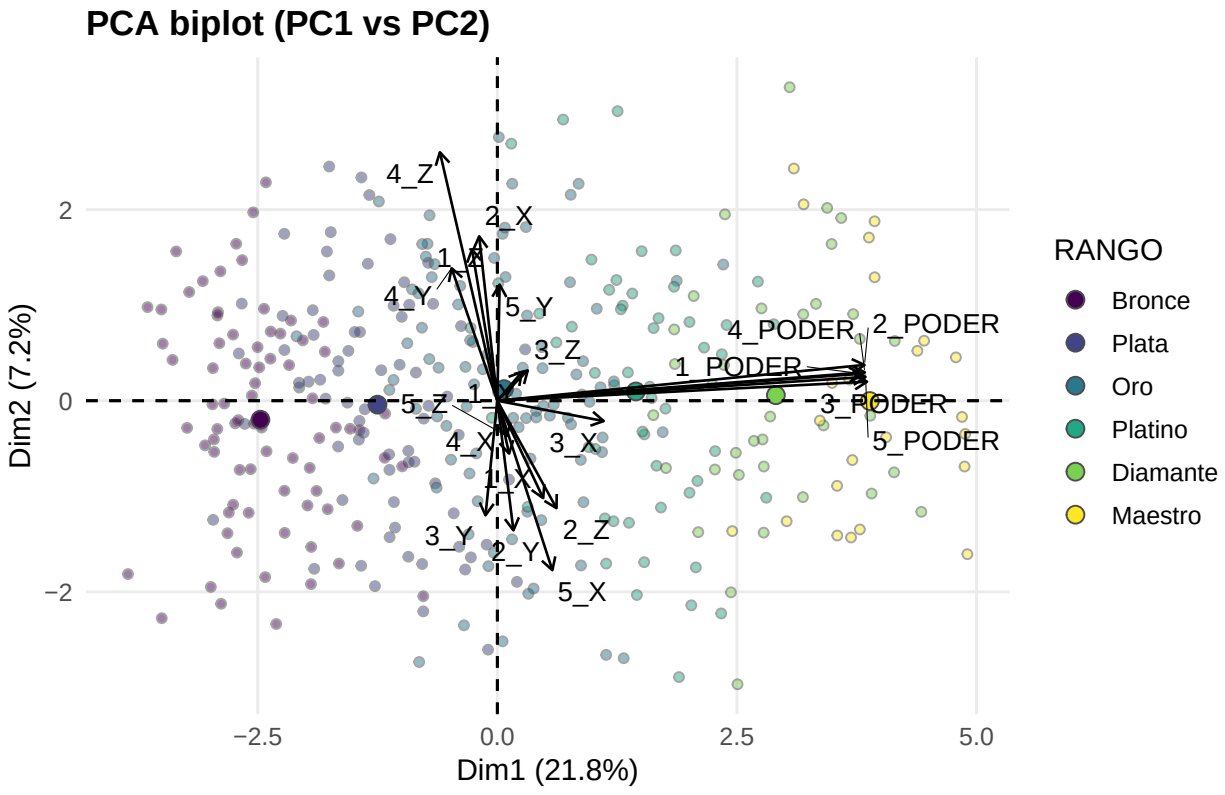


Figure 8: PCA biplot on the first two components, coloured by difficulty rank.

The earlier interpretation of the loadings can now be seen in the plot. The (1_PODER, …, 5_PODER) variables are the most influential, reflected in vector length. The previous feature-importance plot also highlighted these as the most relevant variables. With the first two components

the categories separate to some extent into regions of the 2D space. That segmentation would be clearer if cumulative variance were higher; here it is only 29%.

4 Genetic Algorithm for Feature Selection

Given the sample imbalance, an objective that accounts for different error types is appropriate; hence macro F1 is chosen.

```
#> Best fitness (macro F1): 0.6706
```

```
#> Number of variables selected: 9 of 20
```

```
#> Selected variables:
```

```
#> [1] "1_X"     "2_Z"     "2_PODER" "3_X"     "4_X"     "4_PODER" "5_Y"
#> [8] "5_Z"     "5_PODER"
```

Table 8: Best chromosome (1 = variable selected)

Variable	Selected
1_X	1
1_Y	0
1_Z	0
1_PODER	0
2_X	0
2_Y	0
2_Z	1
2_PODER	1
3_X	1
3_Y	0
3_Z	0
3_PODER	0
4_X	1
4_Y	0
4_Z	0
4_PODER	1
5_X	0
5_Y	1
5_Z	1
5_PODER	1

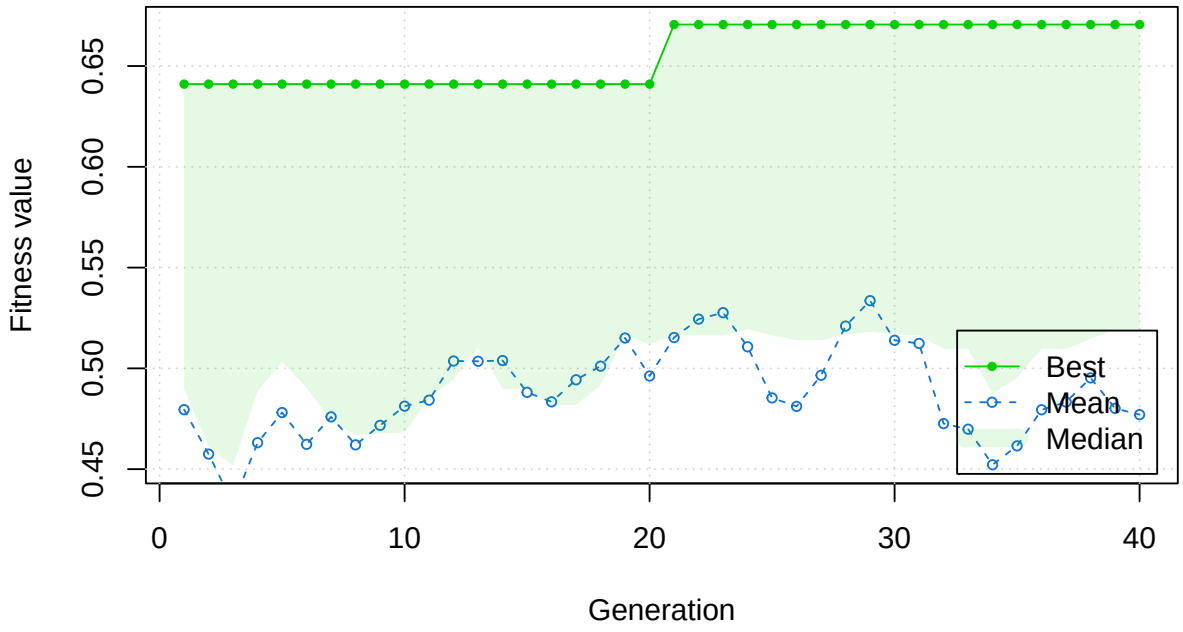


Figure 9: GA fitness evolution across generations (best and mean).

The quality of this feature set will be checked by retraining the network.

Table 9: Neural-network test-set metrics

Model	Accuracy	Balanced_Accuracy	Precision	Recall	F1
Neural Network - GA subset	0.57	0.745	0.577	0.578	0.571

Compared with the earlier results, slight metric improvements are visible.

5 Dichotomous variable DIFICIL

Using the ordinal structure given to the target at the start, the dichotomous variable DIFICIL can now be built relatively easily.

```
#>           DIFICIL
#> RANGO      No_Dificil Dificil
#>   Bronce           76       0
#>   Plata            72       0
#>   Oro              90       0
#>   Platino          53       0
#>   Diamante          0      35
#>   Maestro           0      24
```

5.1 Class distribution

Table 10: DIFICIL class distribution

Class	Count	Proportion
No_Dificil	291	0.831
Dificil	59	0.169

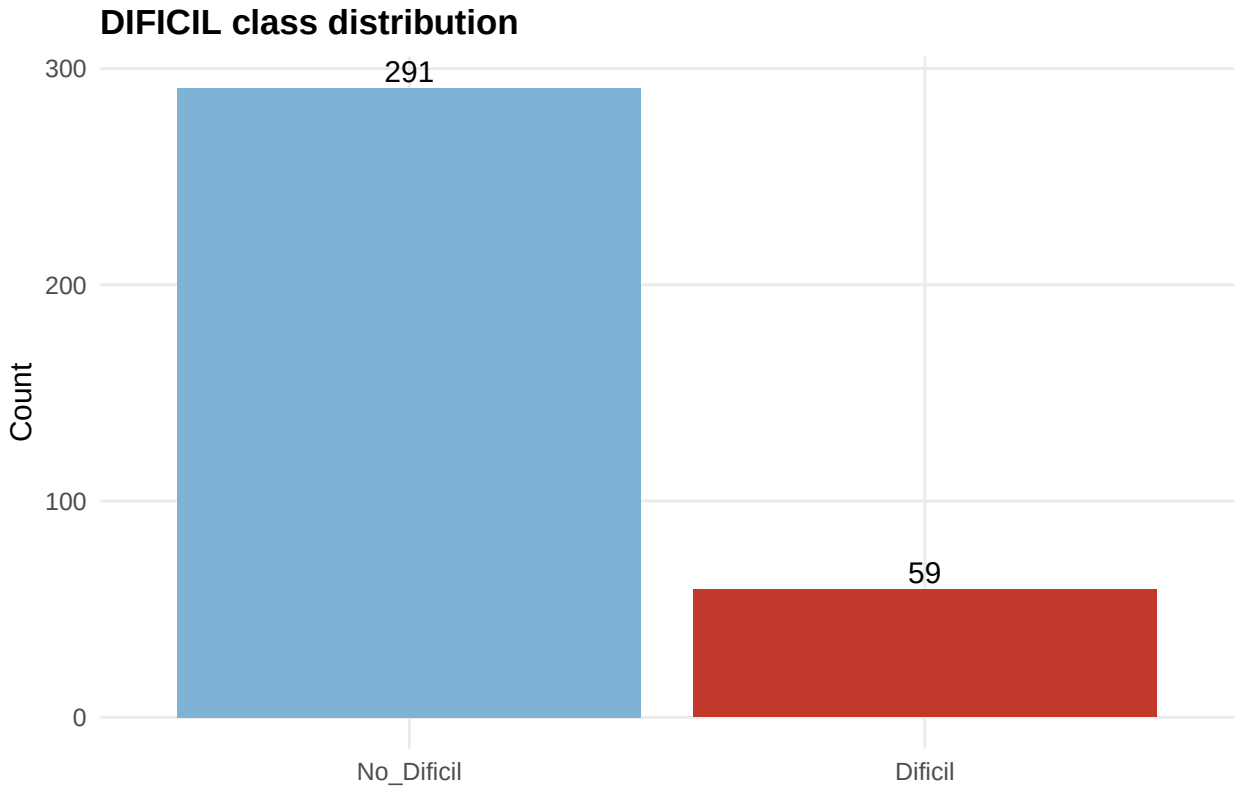


Figure 10: Class distribution of the binary target DIFICIL.

An uneven distribution of the DIFICIL variable is visible.

6 Radial SVM for `DIFICIL`

A cross-validation procedure is used to find the best parameters for the radial SVM.

```
#> Best cost: 100 | Best gamma: 0.001
```

The best parameters yield a small margin and few support vectors; the model will have stiff/global margins and little error tolerance given the high cost.

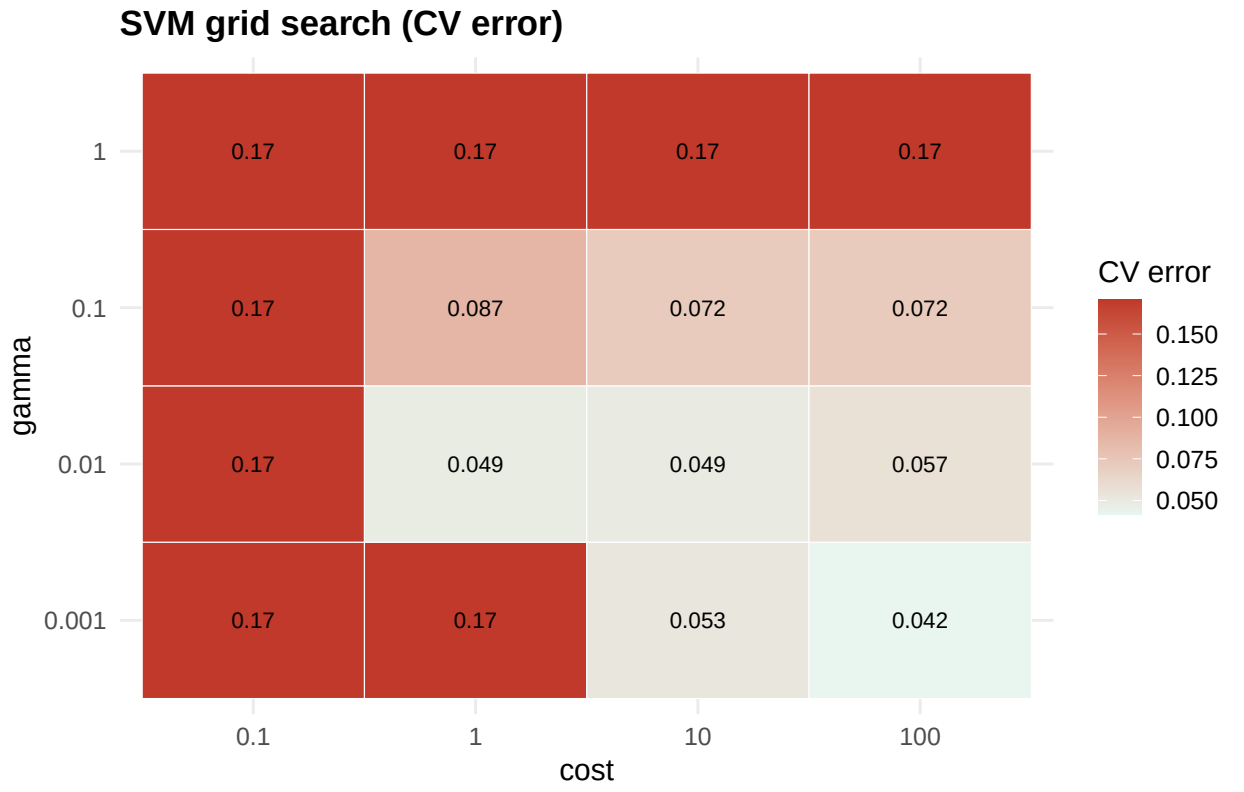
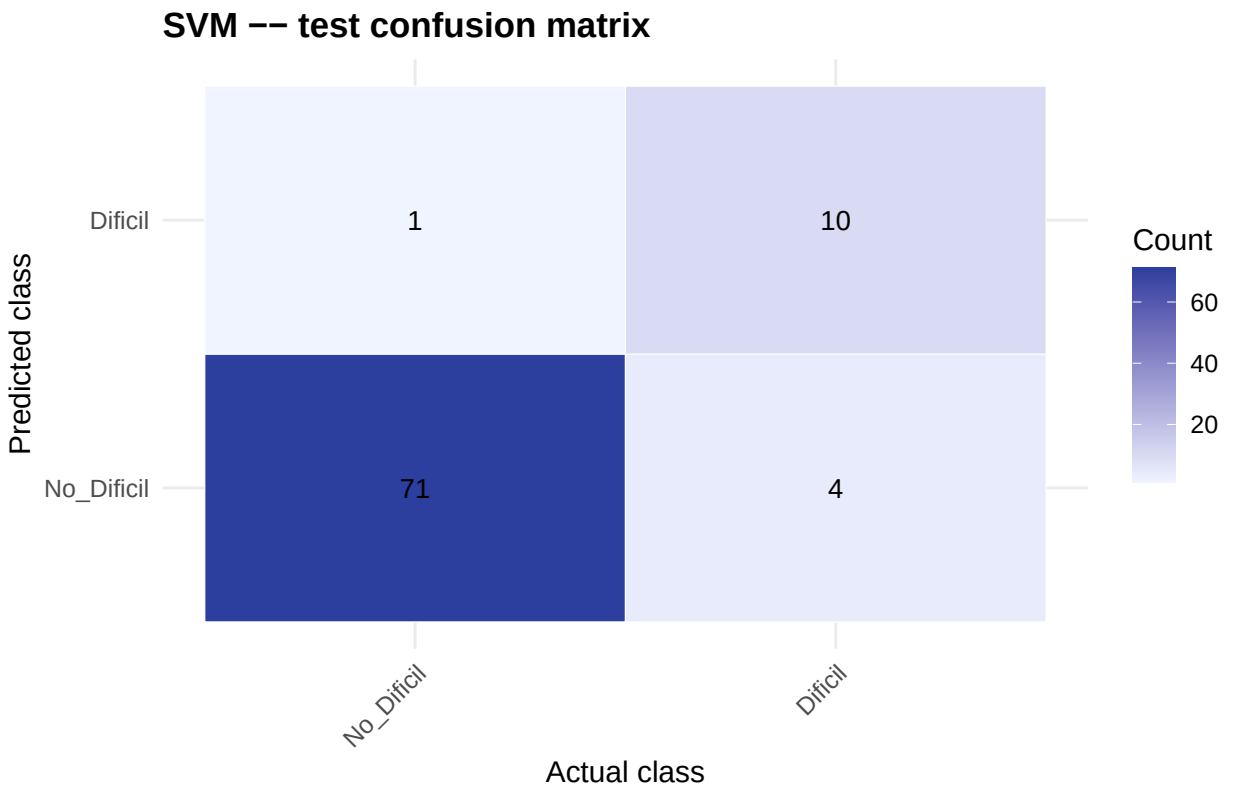


Figure 11: Cross-validation error across the cost/gamma grid.

Table 11: Radial SVM test-set metrics

Model	Accuracy	Balanced_Accuracy	Precision	Recall	F1
Radial SVM (unbalanced)	0.942	0.85	0.909	0.714	0.8



Classification looks good: both the confusion matrix and the metrics are strong. Recall could still improve—i.e. how many “Dificil” observations are labelled “Dificil”.

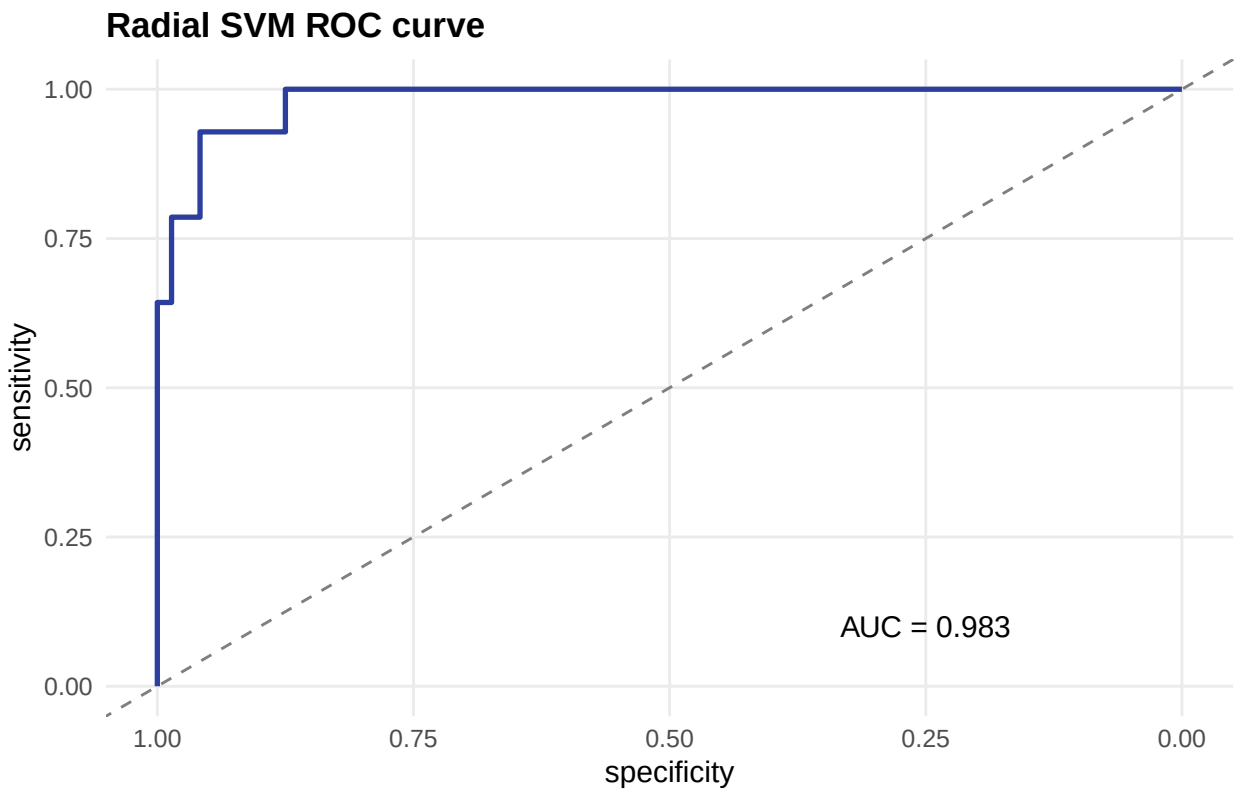


Figure 12: ROC curve for the radial SVM on the test set.

The ROC curve suggests that a better classification threshold is possible. That will be one of the techniques applied in Section 8.

7 The Added Problem

Several possible problems that arise when classifying "Dificil" are examined here.

7.1 Diagnosis 1 – Zero / near-zero variance

Table 12: Near-zero-variance diagnostics (zeroVar / nzv = TRUE is a problem)

	freqRatio	percentUnique	zeroVar	nzv
N_ENEMIES	0.000	0.286	TRUE	TRUE
1_X	1.500	96.857	FALSE	FALSE
1_Y	1.500	92.286	FALSE	FALSE
1_Z	3.600	76.571	FALSE	FALSE
1_PODER	9.500	92.571	FALSE	FALSE
2_X	1.500	96.000	FALSE	FALSE
2_Y	1.500	92.000	FALSE	FALSE

	freqRatio	percentUnique	zeroVar	nzv
2_Z	2.750	75.714	FALSE	FALSE
2_PODER	4.000	93.714	FALSE	FALSE
3_X	1.500	94.571	FALSE	FALSE
3_Y	1.333	92.571	FALSE	FALSE
3_Z	1.500	78.857	FALSE	FALSE
3_PODER	2.000	96.000	FALSE	FALSE
4_X	1.000	92.571	FALSE	FALSE
4_Y	1.500	93.714	FALSE	FALSE
4_Z	3.500	78.000	FALSE	FALSE
4_PODER	2.000	97.714	FALSE	FALSE
5_X	1.000	94.000	FALSE	FALSE
5_Y	1.000	93.143	FALSE	FALSE
5_Z	3.667	80.286	FALSE	FALSE
5_PODER	1.500	97.714	FALSE	FALSE

The only variable with zero variance is N_ENEMIES, which is constant and was already removed from the modelling set.

7.2 Diagnosis 2 – Multicollinearity

```
#> Predictors flagged as highly correlated (|r| > 0.8):
```

```
#> [1] "3_PODER" "4_PODER" "5_PODER"
```

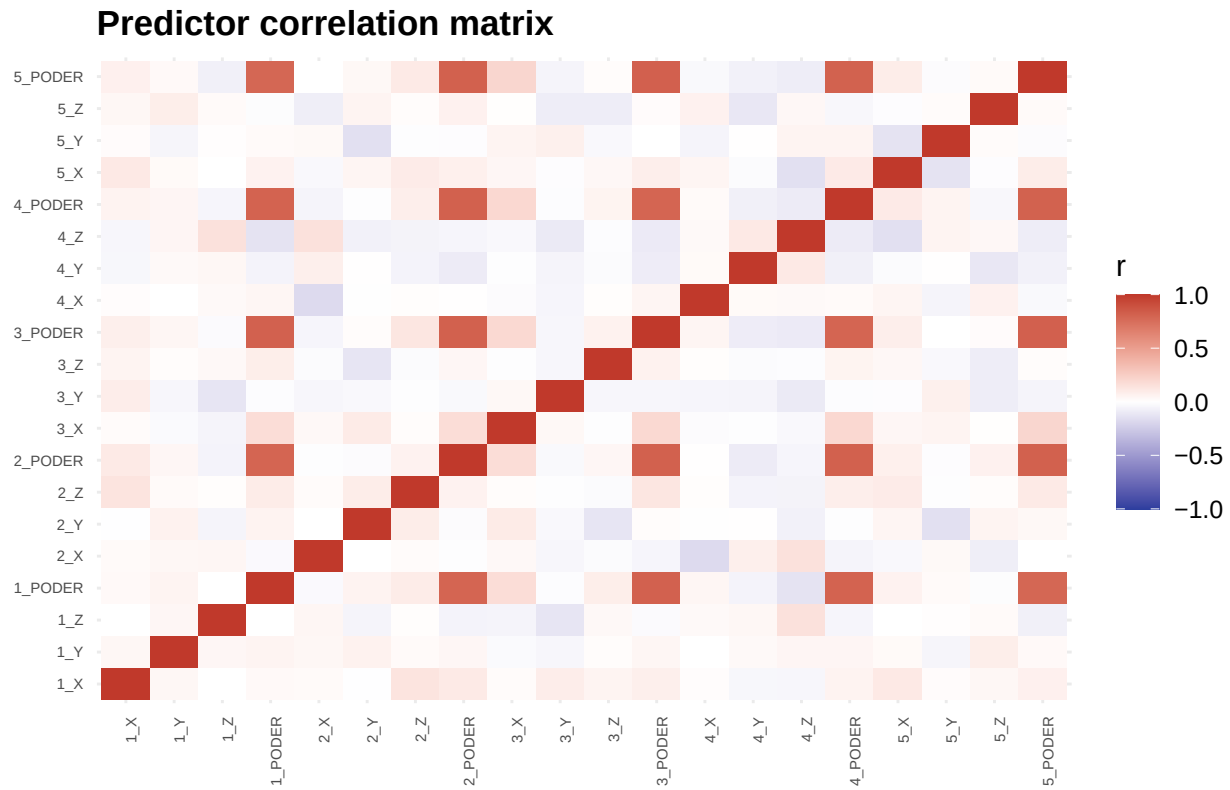


Figure 13: Correlation heat map of the predictors.

Multicollinearity can also hinder model training: redundant information can harm the coefficients associated with these variables.

7.3 Diagnosis 3 – Class imbalance

```
#> DIFICIL class counts:

#>
#> No_Dificil    Dificil
#>        291         59

#> Imbalance ratio (majority : minority) = 4.93 : 1

#>
#> RANGO class counts:

#>
#>   Bronce    Plata      Oro  Platino Diamante  Maestro
#>       76       72       90       53       35       24
```

As seen earlier, the “Dificil” class is unbalanced, so it should be addressed before modelling—via resampling, suitable generalisation metrics, or both.

7.4 Conclusion

Among the problems studied, two may be hindering good model fit: multicollinearity and imbalance in the binary response. For the first, it is enough to drop all but one of the highly collinear variables. For imbalance, several approaches exist:

- Resampling, either oversampling the minority class or undersampling the majority class.
- Using weighted algorithms that redefine the importance of classes during training.
- Adjusting the classification threshold.

8 Balanced Classification

Of the two problems found, class imbalance is addressed first as the more relevant. ROSE resampling is used to balance the sample, and the decision threshold will also be adjusted later.

```
#> Before balancing:
```

```
#>
#> No_Dificil    Dificil
#>        219         45
```

```
#> After balancing (ROSE):
```

```
#>
#> No_Dificil    Dificil
#>        133        131
```

8.1 Balanced SVM

The SVM is retrained on the balanced sample.

```
#> Best cost: 10 | Best gamma: 0.1
```

The hyperparameters changed substantially and are now less extreme, balancing flexibility and error tolerance. Given the small sample, the optimal threshold is computed on the training set so as not to break the required 75%–25% held-out protocol.

	threshold	sensitivity	specificity	accuracy
threshold	0.4988269	1	1	1

The optimal threshold also changes.

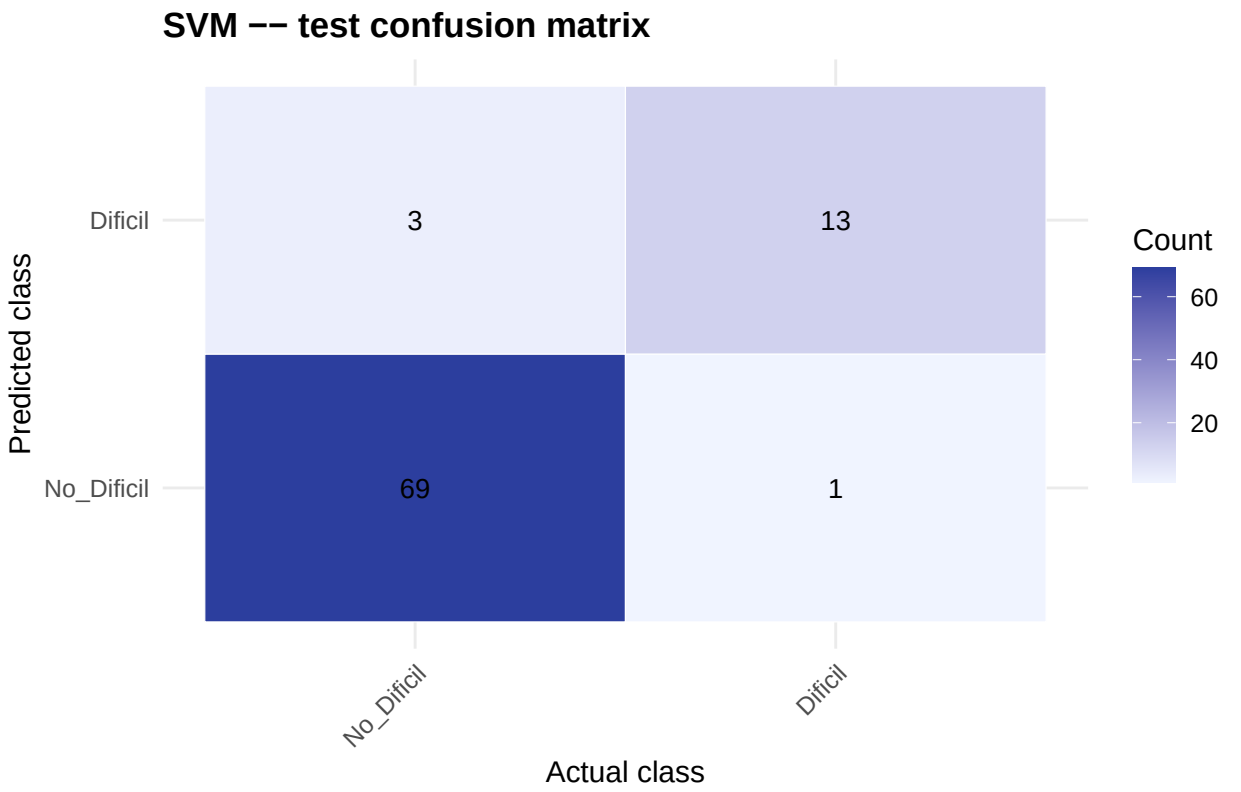


Table 14: Balanced SVM test-set metrics

Model	Accuracy	Balanced_Accuracy	Precision	Recall	F1
SVM (balanced)	0.953	0.943	0.812	0.929	0.867

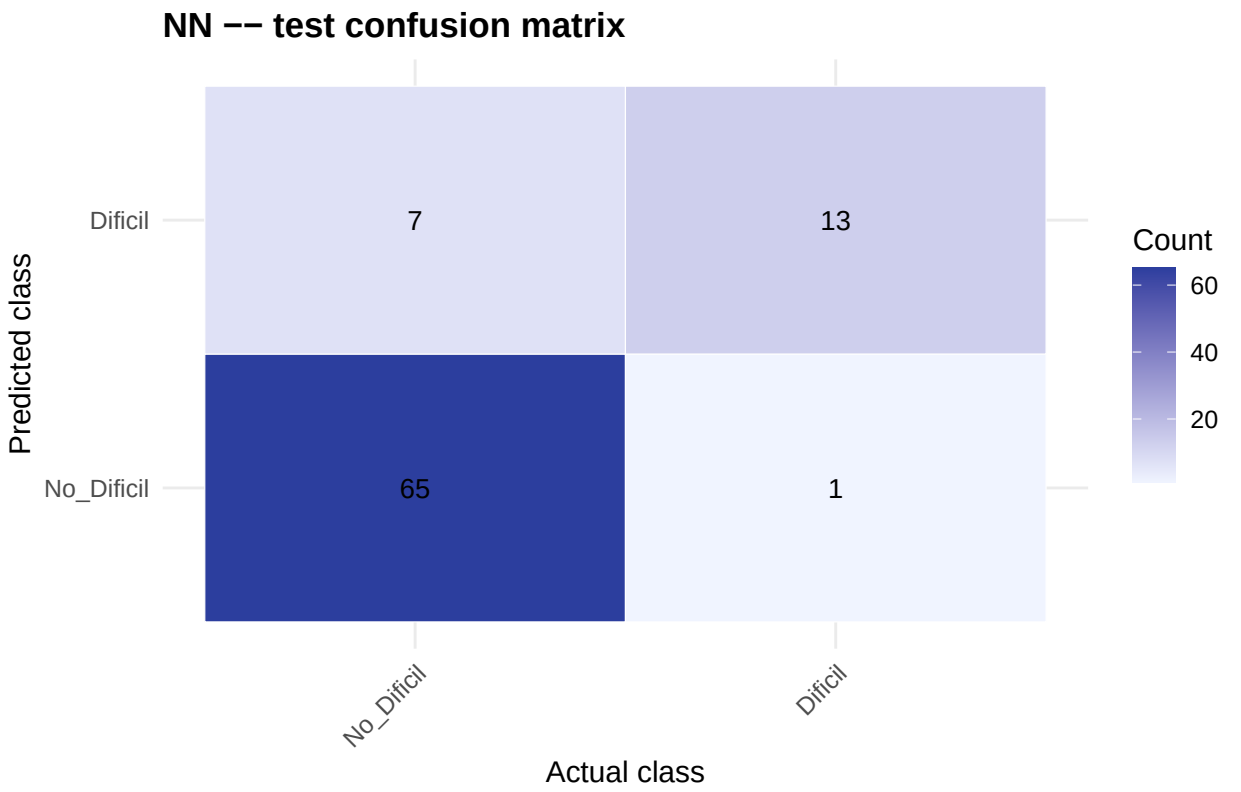
Classification is now better.

8.2 Simple Perceptron

A single-perceptron artificial neural network is fitted.

Table 15: Perceptron test-set metrics

Model	Accuracy	Balanced_Accuracy	Precision	Recall	F1
Perceptron (balanced)	0.907	0.916	0.65	0.929	0.765



8.3 Comparison

Table 16: Binary model comparison (Recall/F1 on 'Dificil' matter most)

Model	Accuracy	Balanced_Accuracy	Precision	Recall	F1
Radial SVM (unbalanced)	0.942	0.850	0.909	0.714	0.800
SVM (balanced)	0.953	0.943	0.812	0.929	0.867
Perceptron (balanced)	0.907	0.916	0.650	0.929	0.765

The balanced SVM classifies the test sample best thanks to the corrections applied, with higher Balanced_Accuracy, Recall and F1; the unbalanced SVM remains a solid candidate. The perceptron does less well, as expected from a single neuron.

8.4 Prediction for `ID = 40`

Table 17: Prediction for dungeon ID = 40

Model	Prediction	Actual
SVM (balanced)	Dificil	No_Dificil
Perceptron (balanced)	Dificil	No_Dificil

Both models mispredict, likely an atypical case. Recall that the perceptron yields a linear boundary while the SVM can bend its boundary locally via γ; even that non-linear behaviour was not enough here.

Reproducibility notes

```
#> R version 4.5.1 (2025-06-13 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 26200)
#>
#> Matrix products: default
#>   LAPACK version 3.12.1
#>
#> locale:
#> [1] LC_COLLATE=Spanish_Spain.utf8  LC_CTYPE=Spanish_Spain.utf8
#> [3] LC_MONETARY=Spanish_Spain.utf8 LC_NUMERIC=C
#> [5] LC_TIME=Spanish_Spain.utf8
#>
#> time zone: Europe/Madrid
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats     graphics  grDevices utils     datasets  methods   base
#>
#> other attached packages:
#>  [1] yardstick_1.4.0      knitr_1.51           ROSE_0.0-4
#>  [4] pROC_1.19.0.1        e1071_1.7-17         GA_3.2.5
#>  [7] iterators_1.0.14     foreach_1.5.2        factoextra_2.0.0
#> [10] nnet_7.3-20          randomForest_4.7-1.2 rpart.plot_3.1.4
#> [13] rpart_4.1.24         caret_7.0-1          lattice_0.22-7
#> [16] lubridate_1.9.5      forcats_1.0.1        stringr_1.6.0
#> [19] dplyr_1.2.1          purrr_1.2.1          readr_2.2.0
#> [22] tidyr_1.3.2          tibble_3.3.0         ggplot2_4.0.3
#> [25] tidyverse_2.0.0
#>
#> loaded via a namespace (and not attached):
#>  [1] rlang_1.1.7          magrittr_2.0.4       otel_0.2.0
#>  [4] compiler_4.5.1       vctrs_0.7.3          reshape2_1.4.5
#>  [7] pkgconfig_2.0.3      crayon_1.5.3         fastmap_1.2.0
#> [10] backports_1.5.0      labeling_0.4.3       rmarkdown_2.30
#> [13] prodlim_2026.03.11   tzdb_0.5.0           tinytex_0.58
#> [16] bit_4.6.0            xfun_0.57            jsonlite_2.0.0
#> [19] recipes_1.4.0        broom_1.0.12         parallel_4.5.1
#> [22] R6_2.6.1             stringi_1.8.7        RColorBrewer_1.1-3
#> [25] parallelly_1.46.1    car_3.1-5            dials_1.4.4
#> [28] Rcpp_1.1.0           future.apply_1.20.2  Matrix_1.7-3
#> [31] splines_4.5.1        timechange_0.4.0     tidyselect_1.2.1
#> [34] abind_1.4-8          yaml_2.3.10          timeDate_4052.112
#> [37] codetools_0.2-20     listenv_0.10.1       plyr_1.8.9
#> [40] withr_3.0.2          S7_0.2.0             evaluate_1.0.5
#> [43] future_1.70.0        survival_3.8-3       proxy_0.4-29
```

```
#> [46] pillar_1.11.1        ggpubr_0.6.3         carData_3.0-6
#> [49] stats4_4.5.1         generics_0.1.4       vroom_1.7.0
#> [52] hms_1.1.4            scales_1.4.0         globals_0.19.1
#> [55] class_7.3-23         glue_1.8.0           tools_4.5.1
#> [58] data.table_1.18.2.1  ModelMetrics_1.2.2.2 gower_1.0.2
#> [61] ggsignif_0.6.4       grid_4.5.1           ipred_0.9-15
#> [64] nlme_3.1-168         DiceDesign_1.10      Formula_1.2-5
#> [67] cli_3.6.5            parsnip_1.6.1        workflows_1.3.0
#> [70] viridisLite_0.4.3    lava_1.8.2           gtable_0.3.6
#> [73] rstatix_0.7.3        digest_0.6.37        ggrepel_0.9.7
#> [76] farver_2.1.2         htmltools_0.5.8.1    lifecycle_1.0.5
#> [79] hardhat_1.4.3        bit64_4.6.0-1        MASS_7.3-65
```

Note. This report is fully reproducible once `dungeons.csv` is placed next to the `.qmd`. Every number, table and figure is produced by executing the code; all statements marked `# FLAG:` must be filled in / confirmed against the rendered output and were intentionally left un-fabricated.

9 Transparency and responsible declaration

While completing the assignment, at least the following changes were made relative to the original code in order to fix errors, improve reproducibility and aid interpretation:

- **Fix to `model_formula` generation.** The original code failed because of variable names (e.g. names starting with a digit). The formula is now built with *backticks*.
- **Adaptation of some models to non-standard variable names.**
  - For **Random Forest**, the formula interface did not accept those names correctly, so the `x_train` / `y_train` interface was used instead.
  - For the **Genetic Algorithm**, the issue persisted, so safe names via `make.names()` were used and `nn_fitness` was updated accordingly.
- **Change to the Random Forest tuning process.** The hyperparameter search was extended to include the number of trees (`ntree`) as well as `mtry`, and later analysis cells were updated accordingly.
- **Improved confusion-matrix displays.** Matrices are now shown systematically as heat maps for clearer interpretation.
- **Explanatory comments.** Comments were added throughout to aid interpretation and keep a clear thread across experiments.
- **Document formatting.** Small formatting changes improved readability and organisation.
- **Simplified neural-network configuration.** `MaxNWts = 5000` was removed as unnecessarily large for this problem size and of no experimental benefit.
- **Change of GA objective.** Macro F1 is used instead, which is more suitable for an unbalanced sample.

- **SVM improvements.** Confusion matrices are shown as heat maps, and a block was added to choose the best ROC-based classification threshold.
- **ROSE formula fix.** The previous formula errored; it was rewritten so `ROSE::ROSE` can parse it.
- **Perceptron code fix.**
- **Threshold classification for the balanced SVM.**
- **Comparison-section fix.** Model changes required updating this section so it runs again.
- **Multiclass metrics function fix.**